SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	DIVYASREE.M
Name	192524098

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**
Question Count: 5

Code of Conduct

I Divyasree.M/192524098 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Divyasree . M*

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. Uneven Input Splits Causing Idle Reducers

Identification of Error	• Data skew in the MapReduce job — a small number of keys hold a disproportionately large share of records, so a few reducers are overloaded while others sit idle. • Poor input split configuration: default block-size-based splitting does not align with the logical distribution of the data, and no custom Partitioner is used. • Excessive small files creating a large number of tiny, uneven splits (the 'small files problem').
Cause Analysis	• The default HashPartitioner sends records to reducers purely by hash of the key, so if one key (e.g., a popular customer ID or a NULL/default value) dominates the dataset, that reducer receives most of the load. • Split size parameters (mapreduce.input.fileinputformat.split.minsize / maxsize) were left at defaults, ignoring the actual size/skew of the source data. • No combiner is used, so all intermediate data — including the skewed key's data — travels to the reducer stage unreduced, worsening the imbalance. • Insufficient number of reducers configured, forcing uneven work onto a small pool of reducer slots.
Corrective Action	• Implement a custom Partitioner (or salting/key-randomization technique) to spread heavy keys across multiple reducers, then aggregate in a second pass. • Introduce a Combiner to perform local aggregation on the map side and reduce the volume of skewed data reaching reducers. • Use CombineFileInputFormat to merge small files into optimally-sized splits and tune split.minsize/split.maxsize to match actual data distribution. • Enable speculative execution so slow (overloaded) reducer tasks are backed up by duplicate tasks on idle nodes. • Right-size the number of reducers based on cluster capacity and expected data volume per key.
Use of Hadoop / Integration Concepts	• InputFormat and Partitioner customization, Combiner functions, CombineFileInputFormat, speculative execution, and data locality principles of the MapReduce execution model.

2. HDFS Data Unavailability After Single Node Failure

Identification of Error	• Replication factor is too low (e.g., set to 1 or 2) or replicas are not distributed across independent failure domains, so losing one node removes the only accessible copy of some blocks. • No NameNode High Availability (HA) configured — the NameNode is a single point of failure for metadata access.
Cause Analysis	• The default replication factor may have been reduced to save storage, leaving critical blocks under-replicated. • Rack awareness (topology script) was not configured, so HDFS's block placement policy could not guarantee replicas land on different racks/nodes — multiple replicas may reside on the same physical node group. • Without a Standby NameNode, JournalNodes, and ZooKeeper Failover Controller, a NameNode/node failure halts metadata access cluster-wide, not just for the failed node's blocks.
Corrective Action	• Restore replication factor to the standard value of 3 (or higher for critical data) via dfs.replication and run the HDFS balancer/replicator to fix under-replicated blocks. • Configure rack awareness so the default block placement policy distributes replicas across racks, ensuring no single node/rack failure causes data loss. • Enable HDFS High Availability with an Active/Standby NameNode pair, shared edits via JournalNodes (Quorum Journal Manager), and ZooKeeper-based automatic failover. • Schedule regular checkpointing of fsimage and edit logs, and monitor block replication health via the NameNode UI/fsck.
Use of Hadoop / Integration Concepts	• HDFS replication, rack-awareness/topology scripts, block placement policy, NameNode HA, Quorum Journal Manager, ZooKeeper Failover Controller, fsimage/edits checkpointing.

3. Duplicate Records from ETL / Apache NiFi Ingestion

Identification of Error	• Lack of idempotency in the ingestion pipeline — the same record can be written more than once to the analytics store. • No deduplication logic in the NiFi flow, combined with at-least-once delivery semantics on retries/failures.
Cause Analysis	• NiFi's provenance-based retry mechanism resends FlowFiles after a transient failure or connection timeout without checking whether the record was already delivered downstream. • The target store lacks unique/primary-key constraints, so re-inserted records are accepted as new rows instead of being rejected or merged. • Parallel processor concurrency (multiple threads processing overlapping FlowFiles) with no shared state to track already-processed record IDs. • The upstream source itself may emit duplicate events (e.g., a message queue with at-least-once guarantees).
Corrective Action	• Insert a DetectDuplicate processor (backed by a Distributed Map Cache) in the NiFi flow to filter out FlowFiles already seen within a configured time window. • Enforce unique keys / primary-key constraints at the destination and use UPSERT (merge-on-conflict) operations instead of plain INSERT. • Track processed record IDs or offsets externally (e.g., Redis, HBase) to make the pipeline idempotent even after reprocessing. • Adopt exactly-once or effectively-once processing patterns: checkpoint offsets, use transactional writes, and validate FlowFile provenance before final load.
Use of Hadoop / Integration Concepts	• Apache NiFi FlowFile provenance, DetectDuplicate processor, Distributed Map Cache, idempotent design, exactly-once semantics, UPSERT/merge patterns in ETL integration.

4. YARN Resource Starvation Under Concurrent Job Submission

Identification of Error	• Improper or default scheduler configuration (e.g., FIFO Scheduler, or a Capacity/Fair Scheduler without defined queues and limits), allowing large jobs to monopolize cluster resources. • No per-user/per-queue resource caps, so a single user's job set can starve all others.
Cause Analysis	• The FIFO Scheduler processes applications strictly in submission order, so an early large job consumes containers cluster-wide while later jobs wait indefinitely. • Even where Capacity or Fair Scheduler is enabled, missing queue definitions (min/max capacity, per-user limits) leave no isolation between users or teams. • Preemption is disabled, so once resources are allocated to a long-running job, they are never reclaimed for higher-priority or fairer distribution.
Corrective Action	• Replace FIFO with the Capacity Scheduler (for guaranteed per-team/queue capacity) or Fair Scheduler (for dynamic fair sharing) depending on organizational needs. • Define queues with explicit minimum/maximum resource shares and per-user limits (yarn.scheduler.capacity.<queue>.maximum-am-resource-percent, user-limit-factor, etc.). • Enable preemption so under-served queues can reclaim resources from over-allocated ones after a grace period. • Apply Dominant Resource Fairness (DRF) to balance CPU and memory allocation fairly across heterogeneous job types.
Use of Hadoop / Integration Concepts	• YARN ResourceManager architecture, Capacity Scheduler vs. Fair Scheduler, queue-based resource isolation, preemption, and Dominant Resource Fairness (DRF).

5. Backup Strategy Restoring Outdated Data

Identification of Error	• The backup/replication design lacks a consistency and recency guarantee — snapshots or replicas used for recovery are stale relative to the latest writes. • No mechanism ensures the most recent, quorum-confirmed version of data is selected during restore.
Cause Analysis	• Backups are taken at long, fixed intervals (full backups only) rather than continuous/incremental capture, so recent writes between backup windows are lost. • The system relies on an eventual-consistency model without version/timestamp tracking, so recovery may pick an older replica that hasn't yet converged. • No write-ahead log (WAL) replication is used to capture in-flight transactions since the last snapshot. • Restore logic does not use quorum-based reads, so it may read from a lagging replica instead of the most up-to-date one.
Corrective Action	• Move to incremental/continuous backups (shorter Recovery Point Objective) instead of infrequent full backups, using tools such as HDFS Snapshots for point-in-time captures. • Maintain a write-ahead log (WAL) and replay it during recovery to bring restored data up to the most recent committed transaction. • Adopt quorum-based consistency for both writes and reads, ensuring recovery always references the majority-confirmed latest version. • Version and timestamp all backups, automate integrity validation after each backup, and define clear RPO/RTO targets to guide backup frequency.
Use of Hadoop / Integration Concepts	• HDFS Snapshot feature, write-ahead logging (WAL), quorum-based consistency, Recovery Point Objective (RPO) / Recovery Time Objective (RTO), incremental backup strategies in distributed storage.